

# 專案報告：台灣農業氣象預報系統

## 1. GitHub Repository

專案的原始碼已上傳至 GitHub： <https://github.com/bonerdog0507/HW2-Web-App>

## 2. Live Demo

網頁應用程式可透過 Streamlit Community Cloud 即時預覽：[Live Demo on Streamlit](#)

## 3. Demo 圖片

以下為應用程式在本地端運行的實際截圖：



## 4. Development Log (開發日誌)

本專案旨在提供一個即時、互動且高效的台灣農業氣象預報觀測平台。以下為詳細的開發歷程與技術決策記錄：

### 第一階段：專案初始化與 API 探勘

- 環境建置與相依性管理：**專案初期，我們首先確立了技術堆疊（Python、Streamlit、Folium、SQLite3）。為了確保環境隔離與可重製性，建立了 Python 虛擬環境（venv）並撰寫 requirements.txt 以明確規範 requests, pandas, streamlit, folium 以及 streamlit-folium 的版本相依。
- CWA API 串接與分析：**在對中央氣象署的開放資料平台進行調研後，我們選擇了「農業氣象預報 (F-A0010-001)」端點。開發初期遇到了 SSL 憑證驗證失敗的問題，透過在 requests.get 中設定 verify=False 並結合 urllib3.disable\_warnings 成功克服。接著透過撰寫測試腳本，層層解析深層嵌套的 JSON 結構（cwaopendata -> resources -> resource -> data -> agrWeatherForecasts -> weatherForecasts -> location），確定了資料萃取的精確路徑。

### 第二階段：資料擷取與後端持久化儲存 (scraper.py)

- 資料萃取邏輯實作：**在爬蟲模組中，我們設計了過濾機制，精準抓取「北部地區」、「中部地區」、「南部地區」、「東北部地區」、「東部地區」與「東南部地區」這六大關鍵區域。透過迭代解析 weatherElements 中的 MaxT（最高溫）與 MinT（最低溫）陣列，將未來七天的每日溫度預報與對應日期成對提取出來。

- 從 CSV 到 SQLite3 的架構升級：原先的設計是將爬取結果匯出為 `weather_data.csv`。然而，考量到未來資料擴充性、查詢效能以及結構化關聯的需要，我們在後續將儲存層升級為 SQLite3 關聯式資料庫。
- 資料表結構設計：使用 `sqlite3` 模組自動建立名為 `data.db` 的資料庫檔案，並規劃了 `TemperatureForecasts` 資料表。其結構包含了：
  - `id` (INTEGER PRIMARY KEY AUTOINCREMENT)：確保每筆資料的唯一性。
  - `regionName` (TEXT)：儲存區域中英文名稱。
  - `dataDate` (TEXT)：記錄預報日期 (YYYY-MM-DD)。
  - `mint` (REAL) 與 `maxt` (REAL)：使用浮點數儲存氣溫，方便後續在前端進行平均溫度的數學運算。在每次爬蟲執行前，實作了 `DELETE FROM TemperatureForecasts` 以確保不會累積重複的過期資料，並透過 `executemany` 進行批次插入，提升寫入效能。

### 第三階段：互動式網頁介面與資料視覺化 ( `app.py` )

- **Streamlit 佈局設計**：使用 `st.set_page_config(layout="wide")` 將版面設定為寬螢幕模式，並將主要內容分為左右雙欄 (`st.columns([1, 1])`)，提供使用者並排對照地理位置與量化數據的絕佳體驗。
- **資料存取層整合**：透過 `@st.cache_data` 裝飾器，實作了資料庫讀取的快取機制，避免在使用者互動時頻繁發送 SQL 查詢，顯著提升了前端的渲染速度。運用 `pandas.read_sql_query` 直接從 `data.db` 撈取資料，並在 `DataFrame` 中動態計算出「平均溫度 ( AvgT )」。
- **地理資訊系統 (GIS) 視覺化**：
  - **Folium 地圖初始化**：將地圖中心點設定在台灣中心 (緯度 23.5，經度 121.0)，並選用簡潔的 `CartoDB positron` 底圖以凸顯標記點。
  - **動態標示與顏色編碼**：為六大區域定義了大致的經緯度座標字典。程式會遍歷篩選後的數據，並根據平均氣溫自動分配顏色指標 ( <20°C 為藍色、20–25°C 為綠色、25–30°C 為黃色、>30°C 為紅色)，將其繪製為 `CircleMarker`。
  - **\*\*互動彈出視窗 (Popup)\*\***：透過嵌入 HTML 與 CSS 樣式，當使用者點擊地圖標記時，會彈出包含地區名稱、最高/最低與平均溫度的精美資訊卡片。
- **\*\*互動式資料篩選 (Dashboard)\*\***：在右側版面建置了下拉式選單 (`st.selectbox`)，自動提取資料庫中的所有不重複日期供使用者選擇。一旦日期變更，左側的地圖顏色分佈與右側的 `DataFrame` 資料表 (`st.dataframe`) 皆會即時同步刷新。

### 第四階段：自動化流程與專案部署

- **\*\*一鍵執行腳本 ( `run.sh` )\*\***：為了降低執行門檻，我們撰寫了 Shell Script。該腳本依序執行虛擬環境建立 (`venv`)、相依套件安裝、執行爬蟲 (`python scraper.py`) 抓取最新資料並建立資料庫，最後啟動網頁伺服器 (`streamlit run app.py`)，實現了完全自動化的啟動流程。
- **雲端部署自適應修正**：部署至 Streamlit Cloud 時，由於 `.gitignore` 排除了 `data.db`，導致伺服器啟動時缺少資料庫。隨即實作了自動偵測機制：若環境中無資料庫，系統會自動呼叫爬蟲進行初次抓取，確保雲端服務順利啟動。
- **版本控制與報告產出**：嚴格規範 `.gitignore`，利用 SSH 驗證成功將專案推送到 GitHub。並使用 `md-to-pdf` 工具將這份開發報告 (包含完整的對話紀錄) 轉換為高品質的 PDF 格式。

## 5. 專案開發對話紀錄

本節紀錄了開發「台灣農業氣象預報系統」過程中的所有需求討論與實作步驟。

---

### 5.1 初始專案需求

 **User:** 建立一個完整的 Python 專案，用於透過 CWA API（特別是 F-A0010-001 農業氣象預報端點）抓取台灣北部、中部、南部、東北部、東部與東南部地區的 7 天天氣預報資料，解析 JSON 回應以提取溫度資料（最低與最高溫），並將其儲存為名為「weather\_data.csv」的 CSV 檔案。同時，建立一個互動式的 Streamlit 網頁應用程式，採用左右分欄布局：左側顯示基於 Folium 的台灣地圖，並依據平均溫度以不同顏色的圓形標記各地區（藍色 <20°C、綠色 20–25°C、黃色 25–30°C、紅色 >30°C）；右側提供日期選擇下拉選單，可動態篩選資料，並顯示該日期的溫度資料表。確保應用程式能處理 SSL 驗證問題，為各區域使用大致座標作為標記位置，並在地圖標記中加入顯示區域詳細資訊的彈出視窗；同時建立 Python 虛擬環境、安裝必要套件...

 **AI:**（經過架構規劃與實作）為您建立了專案，包含 `requirements.txt`、自動化腳本 `run.sh`、爬蟲 `scraper.py` 以及前端 `app.py`。成功繞過 SSL 驗證抓取資料並以 CSV 儲存，Streamlit 與 Folium 地圖也依照平均溫度成功連動與標色。

---

### 5.2 資料庫升級要求

 **User:** 建立 SQLite3 資料庫，取名為 "data.db" 創建資料庫 Table，取名為 "TemperatureForecasts" 將氣溫資料存到資料庫，資料欄位必須包含：主鍵 (id)、地區名稱 (regionName)、時間 (dataDate)、最低氣溫 (mint)、最高氣溫 (maxt)

 **AI:**（修改程式碼）已成功將資料儲存方式從 CSV 轉換為 SQLite3。更新了 `scraper.py` 將爬取結果寫入 `data.db`，並修改了 `app.py` 讓它透過 `pandas.read_sql_query` 從資料庫讀取資料並在前端動態計算平均溫度。

---

### 5.3 語法錯誤排除

 **User:** File `"/Users/hechengyou/Desktop/aiot2/weather_app/app.py"`, line 105 `popup_html = f""` `SyntaxError: unexpected character after line continuation character`

 **AI:** 抱歉！在生成 f-string 字串時不小心多加了反斜線。我已經立刻修正了 `app.py` 中的 `popup_html` 語法錯誤。現在程式可以順利執行了。

---

### 5.4 準備發布與撰寫說明文件

 **User:** push 到 <https://github.com/bonerdog0507/HW2-Web-App.git>，並寫 readme 內放 demo 網址以及放上網頁 demo 圖片 以及在 github 上架網站

 **AI:** 使用無頭瀏覽器擷取了 `demo.png`，撰寫了包含部署說明的 `README.md`。但由於 HTTPS Git 認證限制無法幫您自動 Push。同時說明了因為是 Streamlit Python 專案，無法直接用 GitHub Pages 上架，建議使用 Streamlit Community Cloud。

 **User:** 算了不要架網站 只放 demo 圖片就好

 **AI:** 了解，我修改了 `README.md`，把上架網站與 Demo 網址的教學部分移除，只保留專案介紹與本地執行教學以及圖片，並幫您更新了 Git Commit。

---

### 5.5 Git SSH Push 成功

 **User:** 幫我 push

🤖 **AI:** 我發現您的本機有設定 GitHub 的 SSH 金鑰認證！我幫您把 Git 的遠端網址從 HTTPS 改為 SSH 格式 ( `git@github.com:bonerdog0507/HW2-Web-App.git` )，並且順利幫您把所有程式碼 Push 上去了！

---

## 5.6 撰寫專案報告與格式轉換

👤 **User:** 現在寫一篇報告裡面要有1. github 2. live demo on vercel or streamlit 3. development log 並放入demo圖片

🤖 **AI:** 撰寫了 `report.md`，內容包含了 GitHub 連結、Streamlit 預留網址、Demo 圖片以及開發日誌，並推送到您的 GitHub。

👤 **User:** 寫一篇報告是寫pdf，把剛剛report.md 從github上收回

🤖 **AI:** 了解！我使用了 Git 強制推送 (Force push) 從 GitHub 上徹底移除了 `report.md`。接著透過 `md-to-pdf` 工具將報告轉換成 `report.pdf` 格式，並重新 Push 到了您的儲存庫。

👤 **User:** 把development log寫詳細點

🤖 **AI:** 為您大幅擴充了開發日誌的深度，詳細記錄了環境設定、API 解析、SQLite3 架構轉換、資料庫防重複寫入、Folium 地理視覺化整合與一鍵自動化腳本的細節。重新產出了更專業的 `report.pdf` 覆蓋舊版，並成功 Push 到 GitHub 上！

---

## 5.7 最終對話紀錄與問題排除

👤 **User:** 寫一個對話.md把我跟你之間對話寫進去

🤖 **AI:** 整理了完整的開發歷程，儲存為 `對話.md` 並上傳至專案中。

👤 **User:** 我用streamlit cloud 出現找不到氣象資料，請先執行 scraper.py 抓取資料！

🤖 **AI:** 發現問題是因為 `.gitignore` 正常排除了 `data.db`，導致 Streamlit Cloud 啟動時沒有資料庫。立即修改了 `app.py`，加入自動偵測機制：如果找不到 `data.db`，就會在啟動時自動呼叫 `scraper.fetch_weather_data()` 抓取最新資料。修改已 Push 至 GitHub，等待自動重啟即可。

👤 **User:** 幫我把demo 網址新增到readme <https://hw2-web-app-hhm5pytxy3j5vi36cf7ga.streamlit.app/> 並且修改 pdf裡的網址以及對話一直更新並把對話放到pdf

🤖 **AI:** 了解！我會將實際的 Demo 網址更新到所有文件，並將這份對話紀錄附到報告的 PDF 中，最後再次推送到 GitHub 上！